



Wrocław University
of Science and Technology

INFORMATION SYSTEMS MODELING
LAB03 GROUP 3

WroclArts System **– Documentation –**

– Data Storage Structure –

Student

Maxime LUCE
Student ID: 294620

Teacher

Tomasz KUBIK

Academic year 2025/2026 – April 13, 2026

Contents

Introduction	2
1 Database structure	2
1.1 General Overview	2
1.2 Table events	3
1.3 Table organizations	4
1.4 Table users	4
1.5 Table event_registrations	4
1.6 Table event_creations	5
2 Implementation	6
3 Tests	10
3.1 Running the application	10
3.2 Registration of an user	11
3.3 Organization Creation	12
3.4 Event Creation	13
3.5 Event Registration	15
3.5.1 Simple	15
3.5.2 Advanced	17
3.6 Adding an organization to an existing event	18
Appendixes	21
List of Tables	21
List of Figures	21
List of source codes	22



Introduction

This document presents the Data Storage Structure for the *WroclArts* system for the third laboratory of "Information Systems Modelling".

First, we present the *database structure* 1, then some details about the implementation 2 and we expose finally our tests 3.

1 Database structure

1.1 General Overview

The **Figure 1** presents the five implemented tables for the different entity : user, events and organizations and the connection-table such as the registration to event and creation of an event. This structure includes *many-to-many* relationship for the registration and the creation of an event.

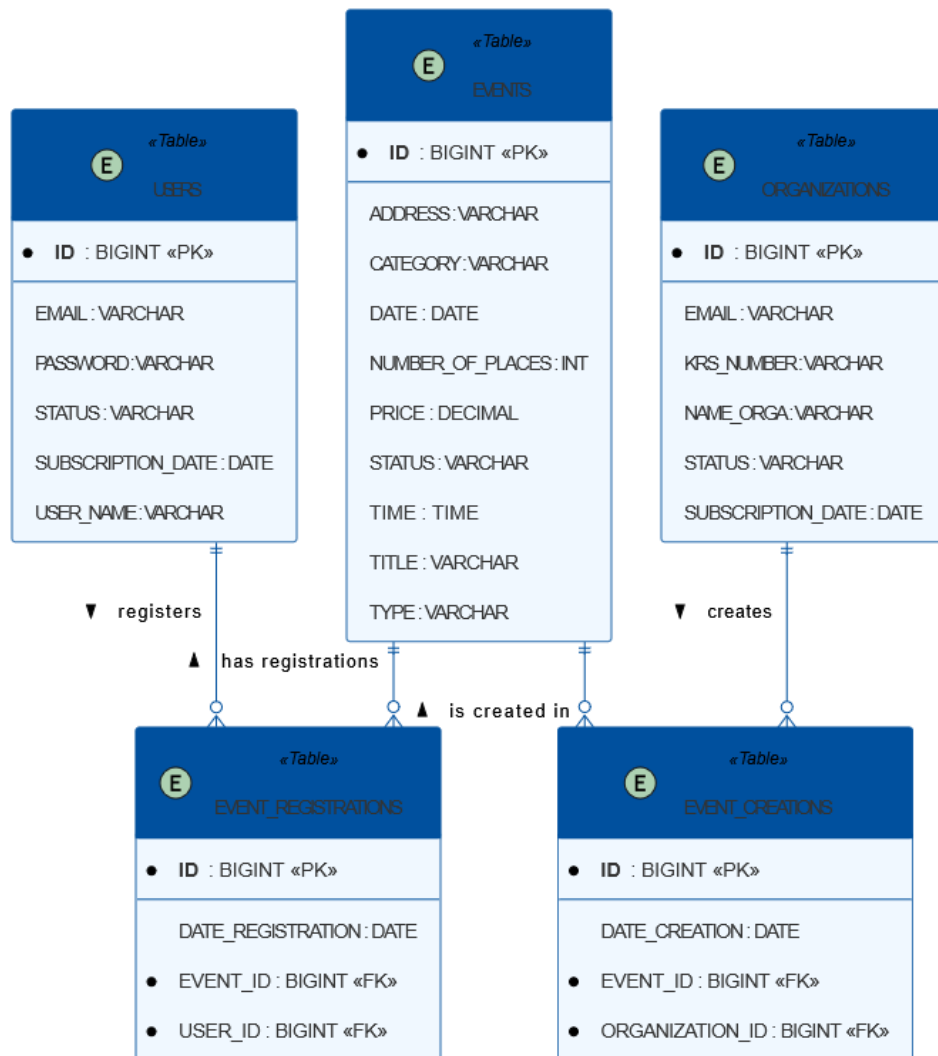


Figure 1: Database structure. We can see the relation *many-to-many* : each user can register to many events and many user cans register to a specific event. Each events is created by one or many organization and an organization can create many events.





1.2 Table events

id	address	category	date	numberOfPlaces	price	status	time	title	type
1	Nadodrze	Crafts	2026-10-14	15	120.0	Published	18:00:00	Pottery Workshop	In-Person
2	Śródmieście, Wrocław	Painting	2026-10-07	12	95.0	Published	17:00:00	Watercolor Painting Class	In-Person
3	Stare Miasto, Wrocław	Drawing	2026-10-10	20	60.0	Published	10:00:00	Urban Sketching Walk	In-Person
4	Nadodrze, Wrocław	Crafts	2026-10-15	10	140.0	Published	16:00:00	Ceramic Sculpture Workshop	In-Person
5	Krzyki, Wrocław	Music	2026-10-18	25	50.0	Published	19:00:00	Choir Singing for Beginners	In-Person
6	Muzeum Współczesne, pl. Wolności, Wrocław	Art Gallery	2026-10-21	18	30.0	Published	14:00:00	Contemporary Art Gallery Tour	In-Person
7	Rynek, Wrocław	Photography	2026-10-24	14	110.0	Published	09:00:00	Street Photography Workshop	In-Person
8	Przedmieście Świdnickie, Wrocław	Drawing	2026-10-28	10	85.0	Published	18:30:00	Portrait Drawing Masterclass	In-Person
9	Fabryczna, Wrocław	Music	2026-11-04	16	70.0	Published	18:00:00	Voice & Vocal Technique Workshop	In-Person
10	Biskupin, Wrocław	Painting	2026-11-08	12	100.0	Published	15:00:00	Acrylic Painting on Canvas	In-Person
11	Olbin, Wrocław	Photography	2026-11-14	8	130.0	Published	17:30:00	Portrait Photography & Lighting	In-Person

Table 1: Description of the table events and fake data within to illustrate the structure. *The fake data were generate with an AI.*

1.3 Table organizations

id	email	krsNumber	nameOrga	status	subscriptionDate
1	contact@ceramicstudio.pl	0000123456	Ceramic Studio Wrocław	PendingVerification	2026-04-10
2	contact@artworkshop-wroclaw.pl	0000654321	ArtWorkshop Wrocław	PendingVerification	2026-04-11

Table 2: Description of the table `organizations` and fake data within to illustrate the structure.

1.4 Table users

id	email	password	status	subscriptionDate	userName
1	tomasz.kubik@pwr.edu.pl	psw	ACTIVE	2026-04-11	T_Kubik

Table 3: Description of the table `users` and fake data within to illustrate the structure.

1.5 Table event_registrations

id	dateRegistration	eventId	userId
1	2026-04-11	1	1
2	2026-04-11	9	1

Table 4: Description of the table `event_registrations` and fake data within to illustrate the structure.

1.6 Table `event_creations`

id	dateCreation	eventId	organizationId
1	2026-04-11	1	1
2	2026-04-11	2	1
3	2026-04-11	3	1
4	2026-04-11	4	1
5	2026-04-11	5	1
6	2026-04-11	6	1
7	2026-04-11	7	1
8	2026-04-11	8	1
9	2026-04-11	9	1
10	2026-04-11	10	1
11	2026-04-11	11	1
12	2026-04-11	8	2

Table 5: Description of the table `event_creations` and fake data within to illustrate the structure.

2 Implementation

The implemented code can be found here <https://github.com/MaximeLuce/WroclArts>.

We present one of the main Controller LISTING 1 : the EventsController.

Listing 1: EventsController.java Example of Controller implemented.

```

1 package pl.edu.pwr.tkubik.ism.api;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.http.HttpStatus;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.web.bind.annotation.RestController;
7 import pl.edu.pwr.tkubik.ism.model.EventEntity;
8 import pl.edu.pwr.tkubik.ism.service.EventRegistrationService;
9 import pl.edu.pwr.tkubik.ism.service.EventService;
10 import pl.edu.pwr.tkubik.ism.service.EventCreationService;
11 import pl.edu.pwr.tkubik.ism.service.OrganizationService;
12 import pl.edu.pwr.tkubik.ism.api.EventApi;
13 import pl.edu.pwr.tkubik.ism.model.Event;
14
15 import java.time.LocalDate;
16 import java.time.LocalTime;
17 import java.util.ArrayList;
18 import java.util.List;
19
20 @RestController
21 public class EventsController implements EventApi {
22
23     @Autowired
24     private EventService eventService;
25
26     @Autowired
27     private EventRegistrationService eventRegistrationService;
28
29     @Autowired
30     private EventCreationService eventCreationService;
31
32     @Autowired
33     private OrganizationService organizationService;
34
35     // private method to convert an Entity (JPA) to a DTO (OpenAPI)
36     private Event convertToDto(EventEntity entity) {
37         Event dto = new Event();
38         dto.setEventId(entity.getId());
39         dto.setTitle(entity.getTitle());
40
41         // convert LocalDate (DB) to String (DTO)
42         if (entity.getDate() != null) {
43             dto.setDate(entity.getDate().toString());
44         }
45
46         // Convert LocalTime (DB) to String (DTO)
47         if (entity.getTime() != null) {
48             dto.setTime(entity.getTime().toString());
49         }
50     }

```



```

51     dto.setAddress(entity.getAddress());
52     dto.setCategory(entity.getCategory());
53     dto.setPrice(entity.getPrice());
54     dto.setNumberOfPlaces(entity.getNumberOfPlaces());
55     dto.setType(entity.getType());
56     dto.setStatus(entity.getStatus());
57
58     return dto;
59 }
60
61 // POST /events
62 @Override
63 public ResponseEntity<Event> addEvent(Event eventDto) {
64     // database Entity instance
65     EventEntity entity = new EventEntity();
66
67     // mappin DTO to Entity
68     entity.setTitle(eventDto.getTitle());
69
70     if (eventDto.getDate() != null) {
71         entity.setDate(LocalDate.parse(eventDto.getDate()));
72     }
73
74     if (eventDto.getTime() != null) {
75         entity.setTime(LocalTime.parse(eventDto.getTime()));
76     }
77
78     entity.setAddress(eventDto.getAddress());
79     entity.setCategory(eventDto.getCategory());
80     entity.setPrice(eventDto.getPrice());
81     entity.setNumberOfPlaces(eventDto.getNumberOfPlaces());
82     entity.setType(eventDto.getType());
83     entity.setStatus(eventDto.getStatus());
84
85     // call the service to save the event in the DB
86     EventEntity savedEntity = eventService.addEvent(entity);
87
88     // creating the link in the even_creations table
89
90     // we suppose by default that organization with ID=1 create this event
91     pl.edu.pwr.tkubik.ism.model.OrganizationEntity dummyOrg = new pl.edu.pwr.
        tkubik.ism.model.OrganizationEntity();
92     dummyOrg.setId(1L);
93
94     // creating the link
95     pl.edu.pwr.tkubik.ism.model.EventCreation creation = new pl.edu.pwr.tkubik.
        ism.model.EventCreation();
96     creation.setEvent(savedEntity);
97     creation.setOrganization(dummyOrg);
98     creation.setDateCreation(LocalDate.now());
99
100    // save the link in DB
101    eventCreationService.addCreation(creation);
102
103    // retrieve the ID generated by the DB and set it in the DTO
104    eventDto.setEventId(savedEntity.getId());
105
106    // return the final DTO with the 200 HTTP status

```




```

107         return new ResponseEntity<>(eventDto, HttpStatus.OK);
108     }
109
110     // POST /events/{eventId}/organizations/{orgId}
111     @Override
112     public ResponseEntity<Event> addOrganizationToEvent(Long eventId, Long orgId) {
113
114         // check if the event exists
115         EventEntity event = eventService.findEventById(eventId);
116         if (event == null) {
117             return new ResponseEntity<>(HttpStatus.NOT_FOUND);
118         }
119
120         // check if the organization exists
121         pl.edu.pwr.tkubik.ism.model.OrganizationEntity organization =
            organizationService.findOrganizationById(orgId);
122         if (organization == null) {
123             return new ResponseEntity<>(HttpStatus.NOT_FOUND);
124         }
125
126         // create the new link in the table "event_creations"
127         pl.edu.pwr.tkubik.ism.model.EventCreation creation = new pl.edu.pwr.tkubik.
            ism.model.EventCreation();
128         creation.setEvent(event);
129         creation.setOrganization(organization);
130         creation.setDateCreation(LocalDate.now());
131
132         // save in DB
133         eventCreationService.addCreation(creation);
134
135         // return the event to confirm
136         return new ResponseEntity<>(convertToDto(event), HttpStatus.OK);
137     }
138
139     // GET /events
140     @Override
141     public ResponseEntity<List<Event>> findAllEvents(Double lat, Double lng, Integer
        radius, String category, String keyword, LocalDate date) {
142         List<EventEntity> entities = eventService.findAllEvents();
143
144         List<Event> dtos = new ArrayList<>();
145         entities.forEach(entity -> dtos.add(convertToDto(entity)));
146
147         return new ResponseEntity<>(dtos, HttpStatus.OK);
148     }
149
150     // GET /events/{eventId}
151     @Override
152     public ResponseEntity<Event> findEventById(Long eventId) {
153         EventEntity entity = eventService.findEventById(eventId);
154
155         if (entity == null) {
156             return new ResponseEntity<>(HttpStatus.NOT_FOUND);
157         }
158
159         return new ResponseEntity<>(convertToDto(entity), HttpStatus.OK);
160     }
161

```



```

162 // PUT /events
163 @Override
164 public ResponseEntity<Event> updateEvent(Event eventDto) {
165     if (eventDto.getEventId() == null) {
166         return new ResponseEntity<>(HttpStatus.BAD_REQUEST);
167     }
168
169     EventEntity existingEntity = eventService.findEventById(eventDto.getEventId()
170         );
171
172     if (existingEntity == null) {
173         return new ResponseEntity<>(HttpStatus.NOT_FOUND);
174     }
175
176     existingEntity.setTitle(eventDto.getTitle());
177
178     if (eventDto.getDate() != null) {
179         existingEntity.setDate(LocalDate.parse(eventDto.getDate()));
180     }
181
182     // Convert String (DTO) to LocalTime (DB)
183     if (eventDto.getTime() != null) {
184         existingEntity.setTime(LocalTime.parse(eventDto.getTime()));
185     }
186
187     existingEntity.setAddress(eventDto.getAddress());
188     existingEntity.setCategory(eventDto.getCategory());
189     existingEntity.setPrice(eventDto.getPrice());
190     existingEntity.setNumberOfPlaces(eventDto.getNumberOfPlaces());
191     existingEntity.setType(eventDto.getType());
192     existingEntity.setStatus(eventDto.getStatus());
193
194     EventEntity updatedEntity = eventService.addEvent(existingEntity);
195
196     return new ResponseEntity<>(convertToDto(updatedEntity), HttpStatus.OK);
197 }
198
199 // DELETE /events/{eventId}
200 @Override
201 public ResponseEntity<Event> deleteEventById(Long eventId) {
202     EventEntity existingEntity = eventService.findEventById(eventId);
203
204     if (existingEntity == null) {
205         return new ResponseEntity<>(HttpStatus.NOT_FOUND);
206     }
207
208     eventService.deleteEventById(eventId);
209
210     return new ResponseEntity<>(convertToDto(existingEntity), HttpStatus.OK);
211 }
212
213 // POST /events/{eventId}/tickets
214 @Override
215 public ResponseEntity<Event> registerForEvent(Long eventId, String body) {
216     // check if the event exists
217     EventEntity event = eventService.findEventById(eventId);
218
219     if (event == null) {

```



```

219         return new ResponseEntity<>(HttpStatus.NOT_FOUND); // Erreur 404
220     }
221
222     // we suppose by default that we use the user with ID=1
223     pl.edu.pwr.tkubik.ism.model.UserEntity dummyUser = new pl.edu.pwr.tkubik.ism.
        model.UserEntity();
224     dummyUser.setId(1L);
225
226     // create the link
227     pl.edu.pwr.tkubik.ism.model.EventRegistration registration = new pl.edu.pwr.
        tkubik.ism.model.EventRegistration();
228     registration.setEvent(event);
229     registration.setUser(dummyUser);
230     registration.setDateRegistration(LocalDate.now());
231
232     // save the link in the table "event_registrations"
233     eventRegistrationService.addRegistration(registration);
234
235     // we return the event to confirm and the HTTP status ok
236     return new ResponseEntity<>(convertToDto(event), HttpStatus.OK);
237 }
238
239 // PUT /tickets/{ticketId}/scan
240 @Override
241 public ResponseEntity<Event> scanTicket(String ticketId, String body) {
242     return new ResponseEntity<>(HttpStatus.NOT_IMPLEMENTED);
243 }
244 }

```

3 Tests

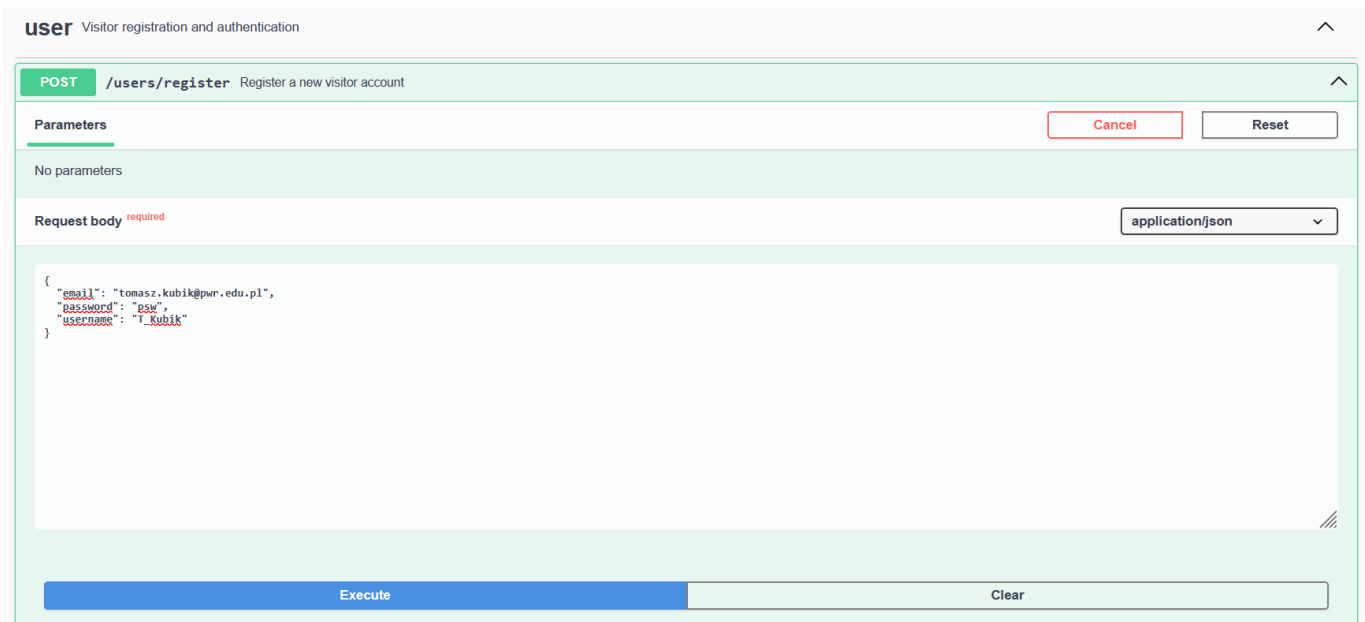
3.1 Running the application

Once the project loaded, you need to proceed as follows:

- Run the paven project with `mvn clean compile`
- Run the application with our IDE
- Test the different routes available at <http://localhost:8080/WroclArts/swagger-ui/index.html>
- The updated data in the database can be seen at <http://localhost:8080/WroclArts/h2-ui/>
 - database: `db_wroclarts (jdbc:h2:file:./db_wroclarts)`
 - username: `sa`
 - password: `(none)`



3.2 Registration of an user



user Visitor registration and authentication

POST /users/register Register a new visitor account

Parameters Cancel Reset

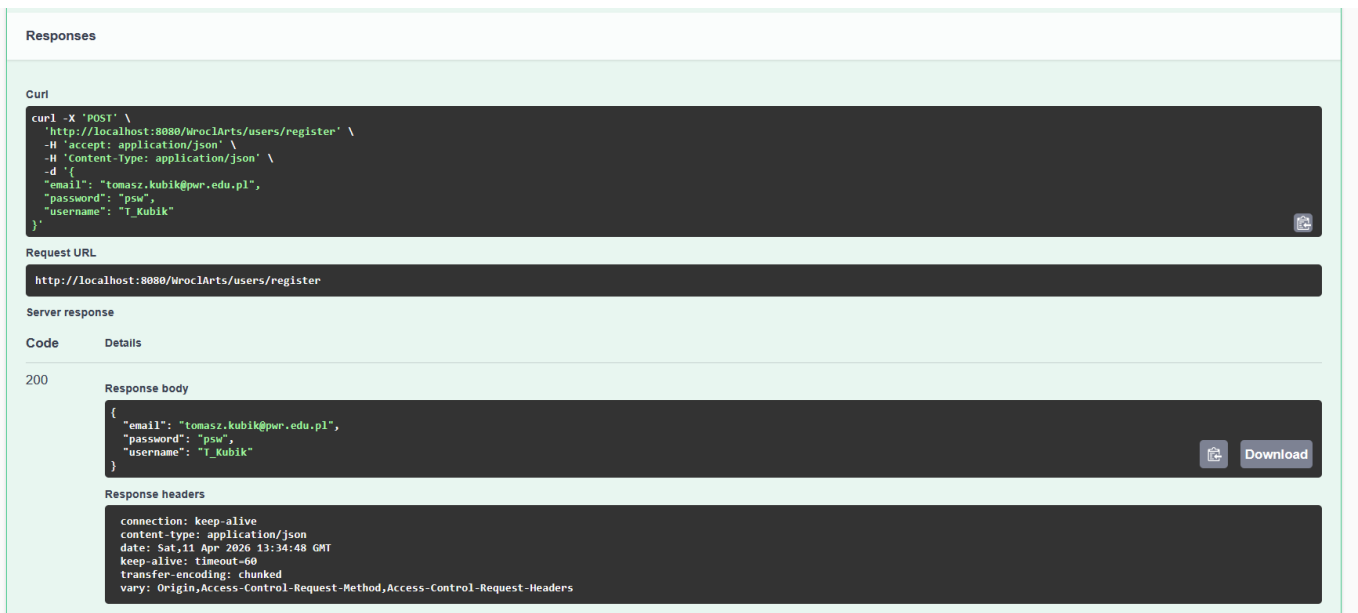
No parameters

Request body required application/json

```
{
  "email": "tomasz.kubik@pwr.edu.pl",
  "password": "psw",
  "username": "T_Kubik"
}
```

Execute Clear

Figure 2: Request POST for the creation of an user. The route is /users/register and the request body contains the required JSON for the entity "User".



Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/WroclArts/users/register' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "email": "tomasz.kubik@pwr.edu.pl",
    "password": "psw",
    "username": "T_Kubik"
  }'
```

Request URL

```
http://localhost:8080/WroclArts/users/register
```

Server response

Code	Details
200	Response body <pre>{ "email": "tomasz.kubik@pwr.edu.pl", "password": "psw", "username": "T_Kubik" }</pre> Download Response headers <pre>connection: keep-alive content-type: application/json date: Sat, 11 Apr 2026 13:34:48 GMT keep-alive: timeout=60 transfer-encoding: chunked vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers</pre>

Figure 3: Response of the backend for the creation of an user. The response contains the status (200, OK) and the JSON of the inserted data of the new entity "User".

Run Run Selected Auto complete Clear SQL statement:

```
SELECT * FROM USERS
```

SELECT * FROM USERS;

ID	EMAIL	PASSWORD	STATUS	SUBSCRIPTION_DATE	USER_NAME
1	tomasz.kubik@pwr.edu.pl	psw	ACTIVE	2026-04-11	T_Kubik

(1 row, 1 ms)

Edit

Figure 4: Inserted data into the database, screenshot of H2 Console. The new user is registered in the table users.

3.3 Organization Creation

POST /organizations register a new organization

Parameters Cancel Reset

No parameters

Request body ^{required} application/json

```
{
  "email": "contact@ceramicstudio.pl",
  "nameOrga": "Ceramic Studio Wrocław",
  "krNumber": "0000123456",
  "subscriptionDate": "2026-04-10",
  "status": "PendingVerification"
}
```

Execute Clear

Figure 5: Request POST for the creation of an user. The route is /organizations and the request body contains the required JSON for the entity "Organization".

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/WroclArts/organizations' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "contact@ceramicstudio.pl",
    "nameOrga": "Ceramic Studio Wroclaw",
    "krsNumber": "0000123456",
    "subscriptionDate": "2026-04-10",
    "status": "PendingVerification"
  }'
```

Request URL

http://localhost:8080/WroclArts/organizations

Server response

Code Details

200

Response body

```
{
  "orgId": 1,
  "email": "contact@ceramicstudio.pl",
  "nameOrga": "Ceramic Studio Wroclaw",
  "krsNumber": "0000123456",
  "subscriptionDate": "2026-04-10",
  "status": "PendingVerification"
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Sat, 11 Apr 2026 13:36:04 GMT
keep-alive: timeout=60
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
```

Figure 6: Response of the *backend* for the creation of an organization. The response contains the status (200, OK) and the JSON of the inserted data of the new entity "Organization".

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM ORGANIZATIONS

SELECT * FROM ORGANIZATIONS;

ID	EMAIL	KRS_NUMBER	NAME_ORGA	STATUS	SUBSCRIPTION_DATE
1	contact@ceramicstudio.pl	0000123456	Ceramic Studio Wroclaw	PendingVerification	2026-04-10

(1 row, 10 ms)

Edit

Figure 7: Inserted data into the database, screenshot of H2 Console. The new user is registered in the table organizations.

3.4 Event Creation

We suppose that the default organization that creates the event has the `id=1` (see LISTING 2).

Listing 2: Details on the implementation of the event creation: we assume by default the organization with `id 1` creates the event.

```
1
2 // Simulation of the Organization (ID 1) creating this event
```



```

3 pl.edu.pwr.tkubik.ism.model.OrganizationEntity dummyOrg = new pl.edu.pwr.tkubik.ism.
  model.OrganizationEntity();
4 dummyOrg.setId(1L);

```

POST /events add a new event

Parameters

No parameters

Request body required application/json

```

{
  "title": "Pottery Workshop",
  "date": "2026-10-14",
  "time": "18:00",
  "address": "Nadodrze",
  "category": "Crafts",
  "price": 120,
  "numberOfPlaces": 15,
  "type": "In-Person",
  "status": "Published"
}

```

Execute

Figure 8: Request POST for the creation of an user. The route is `/events` and the request body contains the required JSON for the entity "Event".

Responses

Curl

```

curl -X 'POST' \
  'http://localhost:8080/WroclArts/events' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Pottery Workshop",
    "date": "2026-10-14",
    "time": "18:00",
    "address": "Nadodrze",
    "category": "Crafts",
    "price": 120,
    "numberOfPlaces": 15,
    "type": "In-Person",
    "status": "Published"
  }'

```

Request URL

`http://localhost:8080/WroclArts/events`

Server response

Code	Details
200	Response body <pre> { "eventId": 1, "title": "Pottery Workshop", "date": "2026-10-14", "time": "18:00", "address": "Nadodrze", "category": "Crafts", "price": 120, "numberOfPlaces": 15, "type": "In-Person", "status": "Published" } </pre> Response headers <pre> connection: keep-alive content-type: application/json date: Sat, 11 Apr 2026 13:38:55 GMT keep-alive: timeout=60 transfer-encoding: chunked vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers </pre>

Figure 9: Response of the *backend* for the registration to an event. The response contains the status (200, OK) and the JSON of the inserted data of the new entity "Event".

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM EVENTS

SELECT * FROM EVENTS;

ID	ADDRESS	CATEGORY	DATE	NUMBER_OF_PLACES	PRICE	STATUS	TIME	TITLE	TYPE
1	Nadodrze	Crafts	2026-10-14	15	120.0	Published	18:00:00	Pottery Workshop	In-Person

(1 row, 2 ms)

Edit

Figure 10: Inserted data into the database, screenshot of H2 Console. The new user is registered in the table events.

3.5 Event Registration

3.5.1 Simple

POST /events/{eventId}/tickets Register/Buy a ticket for an event

Parameters

Cancel

Name	Description
eventId * required integer(\$int64) (path)	1

Request body

application/json

"string"

Execute

Figure 11: Request POST for the registration of an event. The route is /events/{eventId}/tickets and the request body contains additional string data to qualify the event (not implemented yet).

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/WroclArts/events/1/tickets' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"string":' \
```

Request URL

http://localhost:8080/WroclArts/events/1/tickets

Server response

Code Details

200

Response body

```
{
  "eventId": 1,
  "title": "Pottery Workshop",
  "date": "2026-10-14",
  "time": "18:00",
  "address": "Nadodrze",
  "category": "Crafts",
  "price": 120,
  "numberOfPlaces": 15,
  "type": "In-Person",
  "status": "Published"
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Sat, 11 Apr 2026 13:45:26 GMT
keep-alive: timeout=60
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
```

Figure 12: Response of the *backend* for the registration to an event. The response contains the status (200, OK) and the JSON of the data of the selected "Event".

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM EVENT REGISTRATIONS

SELECT * FROM EVENT_REGISTRATIONS;

ID	DATE_REGISTRATION	EVENT_ID	USER_ID
1	2026-04-11	1	1

(1 row, 0 ms)

Edit

Figure 13: Inserted data into the database, screenshot of H2 Console. The new registration is registered in the table event_registrations.

3.5.2 Advanced

POST /events/{eventId}/tickets Register/Buy a ticket for an event

Parameters

Name	Description
eventId * required integer(\$int64) (path)	9

Request body

application/json

"string"

Execute Clear

Figure 14: Request POST for the registration of an event in an advanced case. The route is `/events/{eventId}/tickets` and the request body contains additional string data to qualify the event (not implemented yet).

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/WroclArts/events/9/tickets' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '"string'"
```

Request URL

http://localhost:8080/WroclArts/events/9/tickets

Server response

Code	Details
200	Response body <pre>{ "eventId": 9, "title": "Voice & Vocal Technique Workshop", "date": "2026-11-04", "time": "18:00", "address": "Fabryczna, Wrocław", "category": "Music", "price": 70, "numberOfPlaces": 16, "type": "In-Person", "status": "Published" }</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Sat, 11 Apr 2026 13:51:01 GMT keep-alive: timeout=60 transfer-encoding: chunked vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers</pre>

Figure 15: Response of the *backend* for the creation of an event in an advanced case. The response contains the status (200, OK) and the JSON of the data of the selected "Event".

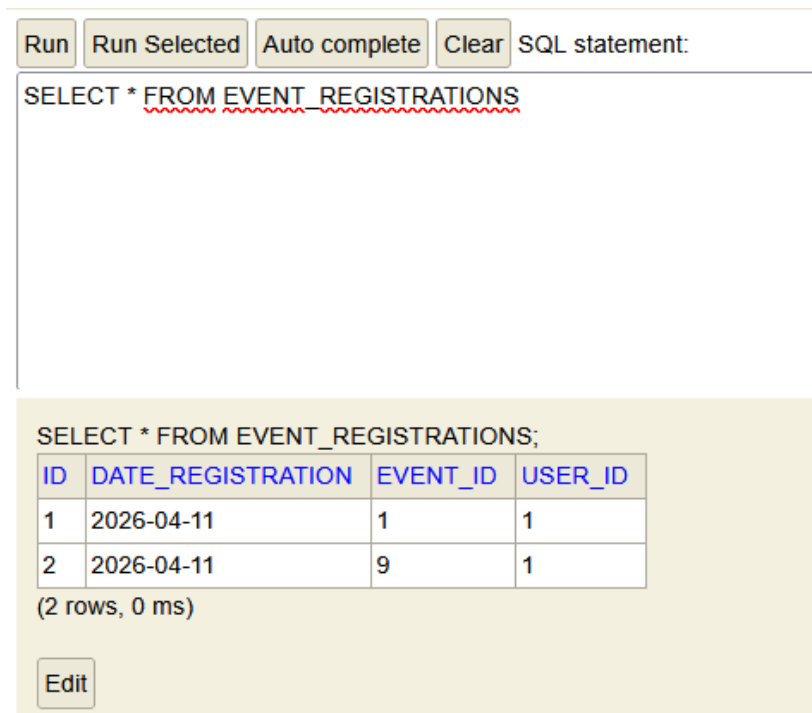


Figure 16: Inserted data into the database, screenshot of H2 Console. The new registration in an advanced case is registered in the table `event_registrations`.

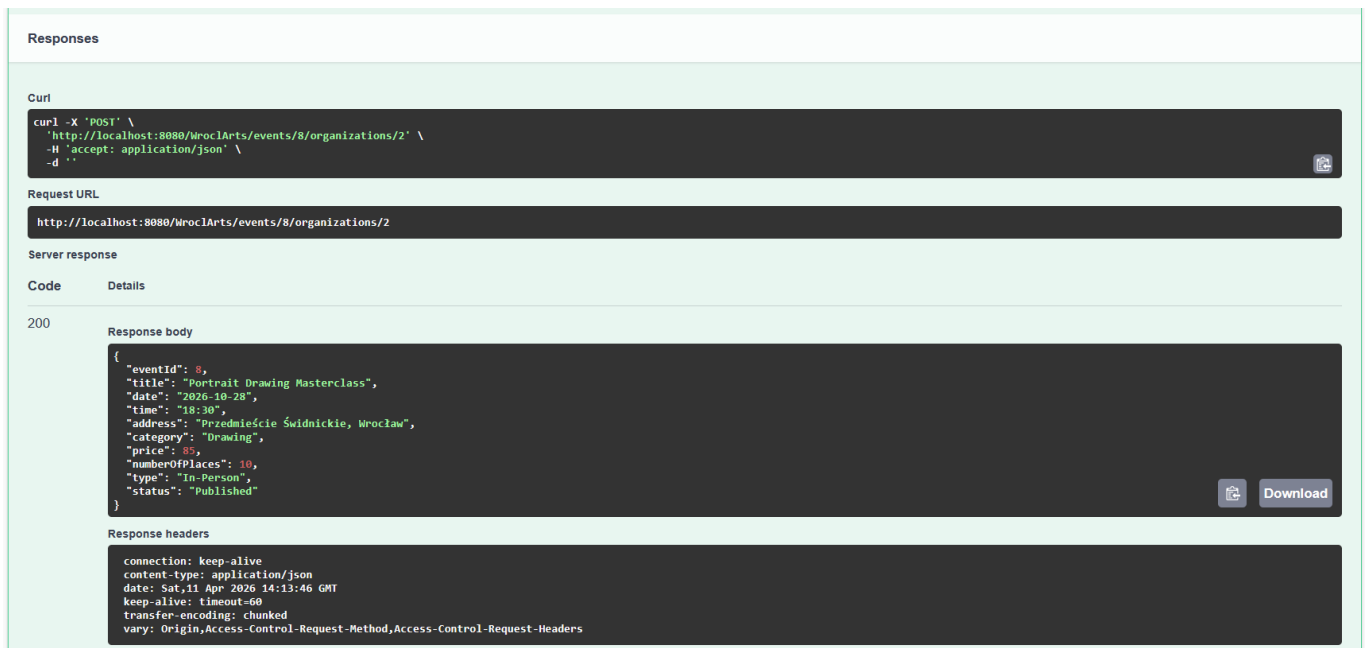
3.6 Adding an organization to an existing event

The screenshot shows a REST client interface for a POST request. The URL is `/events/{eventId}/organizations/{orgId}` with the description 'Add an organization to an existing event (Co-host)'. The 'Parameters' tab is active, showing two required parameters:

- eventId** (required): `integer($int64)` (path), value: `8`
- orgId** (required): `integer($int64)` (path), value: `2`

At the bottom, there is a large blue 'Execute' button and a red 'Cancel' button in the top right corner.

Figure 17: Request POST for the adding of an organization to an existing event. The route is `/events/{eventId}/organizations/{orgId}` and the request body contains the `eventId` and the `orgId`.



Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/WroclArts/events/8/organizations/2' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8080/WroclArts/events/8/organizations/2
```

Server response

Code	Details
200	Response body <pre>{ "eventId": 8, "title": "Portrait Drawing Masterclass", "date": "2026-10-28", "time": "18:30", "address": "Przedmieście Świdnickie, Wrocław", "category": "Drawing", "price": 80, "numberOfPlaces": 10, "type": "In-Person", "status": "Published" }</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Sat, 11 Apr 2026 14:13:46 GMT keep-alive: timeout=60 transfer-encoding: chunked vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers</pre>

Figure 18: Response of the *backend* for the adding of an organization to an existing event. The response contains the status (200, OK) and the JSON of the data of the selected "Event".

Run Run Selected Auto complete Clear SQL statement:

SELECT * FROM EVENT CREATIONS

SELECT * FROM EVENT_CREATIONS;

ID	DATE_CREATION	EVENT_ID	ORGANIZATION_ID
1	2026-04-11	1	1
2	2026-04-11	2	1
3	2026-04-11	3	1
4	2026-04-11	4	1
5	2026-04-11	5	1
6	2026-04-11	6	1
7	2026-04-11	7	1
8	2026-04-11	8	1
9	2026-04-11	9	1
10	2026-04-11	10	1
11	2026-04-11	11	1
12	2026-04-11	8	2

(12 rows, 0 ms)

Edit

Figure 19: Inserted data into the database, screenshot of H2 Console. The new organization is registered in the table event_creations.

Appendixes

List of Tables

1	Description of the table <code>events</code> and fake data within to illustrate the structure. <i>The fake data were generate with an AI.</i>	3
2	Description of the table <code>organizations</code> and fake data within to illustrate the structure.	4
3	Description of the table <code>users</code> and fake data within to illustrate the structure.	4
4	Description of the table <code>event_registrations</code> and fake data within to illustrate the structure.	4
5	Description of the table <code>event_creations</code> and fake data within to illustrate the structure.	5

List of Figures

1	Database structure. We can see the relation <i>many-to-many</i> : each user can register to many events and many user cans register to a specific event. Each events is created by one or many organization and an organization can create many events.	2
2	Request POST for the creation of an user. The route is <code>/users/register</code> and the request body contains the required JSON for the entity "User".	11
3	Response of the <i>backend</i> for the creation of an user. The response contains the status (200, OK) and the JSON of the inserted data of the new entity "User".	11
4	Inserted data into the database, screenshot of H2 Console. The new user is registered in the table <code>users</code>	12
5	Request POST for the creation of an user. The route is <code>/organizations</code> and the request body contains the required JSON for the entity "Organization".	12
6	Response of the <i>backend</i> for the creation of an organization. The response contains the status (200, OK) and the JSON of the inserted data of the new entity "Organization". . .	13
7	Inserted data into the database, screenshot of H2 Console. The new user is registered in the table <code>organizations</code>	13
8	Request POST for the creation of an user. The route is <code>/events</code> and the request body contains the required JSON for the entity "Event".	14
9	Response of the <i>backend</i> for the registration to an event. The response contains the status (200, OK) and the JSON of the inserted data of the new entity "Event".	14
10	Inserted data into the database, screenshot of H2 Console. The new user is registered in the table <code>events</code>	15
11	Request POST for the registration of an event. The route is <code>/events/{eventId}/tickets</code> and the request body contains additional string data to qualify the event (not implemented yet).	15
12	Response of the <i>backend</i> for the registration to an event. The response contains the status (200, OK) and the JSON of the data of the selected "Event".	16



13	Inserted data into the database, screenshot of H2 Console. The new registration is registered in the table <code>event_registrations</code>	16
14	Request POST for the registration of an event in an advanced case. The route is <code>/events/{eventId}/tickets</code> and the request body contains additional string data to qualify the event (not implemented yet).	17
15	Response of the <i>backend</i> for the creation of an event in an advanced case. The response contains the status (200, OK) and the JSON of the data of the selected "Event". . .	17
16	Inserted data into the database, screenshot of H2 Console. The new registration in an advanced case is registered in the table <code>event_registrations</code>	18
17	Request POST for the adding of an organization to an existing event. The route is <code>/events/{eventId}/organizations/{orgId}</code> and the request body contains the <code>eventId</code> and the <code>orgId</code>	18
18	Response of the <i>backend</i> for the adding of an organization to an existing event. The response contains the status (200, OK) and the JSON of the data of the selected "Event".	19
19	Inserted data into the database, screenshot of H2 Console. The new organization is registered in the table <code>event_creations</code>	20

List of source codes

1	<code>EventsController.java</code> Example of Controller implemented.	6
2	Details on the implementation of the event creation: we assume by default the organization with <code>id 1</code> creates the event.	13